Two-Stage Deep Reinforcement Learning for Inverter-Based Volt-VAR Control in Active Distribution Networks

Haotian Liu, *Graduate Student Member, IEEE*, and Wenchuan Wu, *Fellow, IEEE*

***Abstract*—Model-based Vol/VAR optimization method is widely used to eliminate voltage violations and reduce network losses. However, the parameters of active distribution networks(ADNs) are not onsite identified, so significant errors may be involved in the model and make the model-based method infeasible. To cope with this critical issue, we propose a novel two-stage deep reinforcement learning (DRL) method to improve the voltage profile by regulating inverter-based energy resources, which consists of offline stage and online stage. In the offline stage, a highly efficient adversarial reinforcement learning algorithm is developed to train an offline agent robust to the model mismatch. In the sequential online stage, we transfer the offline agent safely as the online agent to perform continuous learning and controlling online with significantly improved safety and efficiency. Numerical simulations on IEEE test networks not only demonstrate that the proposed adversarial reinforcement learning algorithm outperforms the state-of-art algorithm, but also show that our proposed two-stage method achieves much better performance than the existing DRL based methods in the online application.**

***Index Terms*—Voltage control, transfer learning, deep reinforcement learning, reactive power.**

I. Introduction

VOLT/VAR control (VVC) has been successfully integrated into distribution management system to optimize the reactive power flow, achieving the goal of eliminating the voltage violations and reducing network losses. Conventionally, VVC is a model-based optimization method to generate a set of optimal strategies for voltage regulation devices and reactive power resources [1].

With an increasing penetration of distributed generations (DG) in active distribution network (ADN), the problems of voltage violations and high network losses are becoming more severe, especially in the case of reversed active power flow [2], [3]. Due to the fact that most DGs are inverter-based energy resources (IB-ER) and typically produce less active power than the rated capacity, it is reasonable and required for the IB-ERs to provide Volt/VAR support as illustrated in [4].

Till now, most VVC methods are model based. This is to say, the problem of VVC has been described as a nonlinear programming problem. The majority of the existing VVC algorithms in both centralized manners or decentralized manners employ various optimization techniques with real-time measurements, which rely on the accurate model of the physical system. The common centralized VVC algorithms include the well-known conic relaxation methods [4], interior point methods [5], mixed integer linear programming [6], and evolutionary algorithms [2], [7]. Also, plenty of literature adopted distributed or decentralized algorithms under different structures. Distributed algorithms such as [8] and [9] usually require no communication with neighbors and may suffer with the optimality of network loss. Decentralized algorithms often trade the convergence for fast control speed, including quasi real-time reactive optimization [10], alternating direction method of multipliers (ADMM) [11], [12] and accelerate ADMM based distributed control [3]. To address the uncertainty issues regarding DERs, robust optimization [13] and scenario based uncertainty optimization [14] are also proposed. Concerning the active and reactive coupling problem, there are also some works [15], [16] utilizing coordinated control to ease the voltage problem.

However, in ADNs, the network parameters are theoretical parameters instead of onsite identified ones, which cannot reflect the real operation states and significant errors are involved [17]–[20]. The model mismatch issue hinders the application of existing model-based methods in the real world. Therefore, model-free optimization is an alternative and promising solution for VVC, which learns optimal actions with only measurements data and continuous exploration in the action space. Legacy model-free optimization methods were mainly applied to the wind farm control using game theory [17] or gradient-based optimization [18]. Reference [19] proposes an extremum seeking based VVC by introducing sinusoidal perturbations to extract gradient information and optimize the distribution network. In recent years, deep reinforcement learning (DRL) algorithms have demonstrated remarkable performance on multiple controlling tasks such as games [21], [22], autonomous driving [23] and continuous control [24].

Hence, DRL-based VVC algorithms have been developed and compared to traditional optimization-based methods

Manuscript received April 24, 2020; revised September 12, 2020 and November 17, 2020; accepted November 26, 2020. Date of publication December 1, 2020; date of current version April 21, 2021. This work was supported in part by the National Natural Science Foundation of China under Grant U2066601 and Grant 51725703. Paper no. TSG-00631-2020. *(Corresponding author: Wenchuan Wu.)*

The authors are with the State Key Laboratory of Power Systems, Department of Electrical Engineering, Tsinghua University, Beijing 100084, China (e-mail: lht18@mails.tsinghua.edu.cn; wuwench@tsinghua.edu.cn).

Color versions of one or more figures in this article are available at https://doi.org/10.1109/TSG.2020.3041620.

Digital Object Identifier 10.1109/TSG.2020.3041620

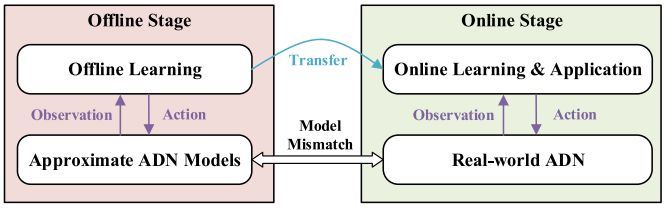


Fig. 1. The scheme of the proposed two-stage VVC algorithm.

in [20], [25]–[29]. In these works, DRL-based VVC methods have shown notable improvement on the performance. For example, [27] uses deep Q learning algorithm to control the slow-timescale discrete capacitors and achieves better voltage profiles. Reference [28] also provides an improved Q learning method for optimal tap setting of load tap changers. In [20], a safe off-policy DRL algorithm based on soft actor-critic (SAC) is proposed and applied to VVC, which results in better voltage profiles and lower power loss comparing with optimization-based methods.

However, the online training start-up of the agents in DRL-based VVC methods can lead to unbearable cost and high risk, since the algorithm has little knowledge of the real system. It is reasonable and desirable for the agents to implement offline training before online training and application. Since the configuration of ADNs are continually updated, historical data is not appropriated for offline training in many scenarios. Therefore, an ADN simulation model is needed to train the agent offline. As mentioned above, the parameters of ADN are inaccurate, so special learning algorithm is indispensable to accommodate this situation. The agents trained on an inaccurate model may show undesirable performance when applied to the real system, which is called the "transfer gap".

In this article, we propose a two-stage deep reinforcement learning method to optimize the reactive power distribution in ADNs, which including offline learning stage and online learning & application stage as shown in fig. 1. In the offline stage, an approximate ADN model using theoretical parameters is built to train a robust agent. Then, the offline trained agent based on model are transferred to online stage. Since we incorporate approximate model information in the offline stage, the online exploration efficiency and safety can be significantly improved for the online stage.

In the offline stage, we propose a novel joint adversarial soft actor-critic (JASAC) algorithm with the formulation of adversarial Markov decision process (AMDP) and view the modeling error in the simulation as an extra disturbance to the system. An adversary agent on behalf of the modeling error and risk is utilized to make the main (i.e., protagonist) agent robust to the model mismatch, which is called the offline agent (OFF-A). JASAC guarantees an efficient convergence of the adversarial training process taking advantage of the shared information between the adversary agent and OFF-A. In the online stage, we implement the online agent (ON-A) with knowledge of OFF-A using the state-of-art DRL algorithm called SAC. Thus, the online training process is significantly accelarated for online application.

Compared with previous studies, the unique contributions of this article are summarized as follows.

1) A two-stage deep reinforcement learning method is proposed to improve the online safety and efficiency via offline pre-training. The proposed data-driven VVC simultaneously works in model-free manner in the online stage and incorporates the knowledge of approximate models in the offline stage to accelerate online learning. This feature makes the proposed method more practical and effective for the real-world applications.
2) In the offline stage, AMDP is formulated and an adversary agent is trained to exploit the modeling errors of ADN. The introduction of adversary agent makes the trained OFF-A robust to the model mismatch. Instead of conducting the adversarial learning separately as the existing works, this article proposes a novel JASAC to exploit the shared information between agents, which endows the algorithm with remarkable higher efficiency and convergence. The proposed JASAC algorithm can also be further applied to other DRL-based application.
3) In the online stage, the OFF-A is transferred to ON-A for continuous learning and application. Since the OFF-A is robust to the transfer gap due to the adversarial learning offline, the proposed algorithm gains significantly better performance and safety in the online application than the legacy DRL-based VVC.

The remainder of this article is organized as follows. Section II formulates the online and offline stage of the VVC problem for ADNs as a MDP and a AMDP respectively. Then, the detailed introduction to the proposed VVC algorithm and JASAC are presented in Section III. In Section IV the results of our numerical study are shown and analyzed. Finally, Section V concludes this article.

II. Preliminaries

In this section, we cover the preliminaries of MDP and AMDP, and then formulate the VVC problem in ADNs into MDP and AMDP.

A. Markov Decision Process and Reinforcement Learning

In the online stage, the RL agent ON-A learns through the interaction with an environment $\mathcal{E}$. As a formalization, MDP defined by the tuple $(\mathcal{S}, \mathcal{A}, p, \mathcal{R}, \gamma)$ is utilized to describe the interaction process and suppose the state of the next transition depends only on current state and the action of the agent. It should be noted that the exact state space $\mathcal{S}$ is often impossible to get in the real world, so we define $\mathcal{O}$ as the observation space for $\mathcal{S}$. Due to standard conventions for notation, we put $s \in \mathcal{S}$ in places instead of $o \in \mathcal{O}$ and treat the unobserved states as noises.

In this article, the state space $\mathcal{S}$ and action space $\mathcal{A}$ are continuous, and the unknown state transition probability $\rho : \mathcal{S} \times \mathcal{S} \times \mathcal{A} \to [0, \infty)$ is the probability density of the next state $s_{t+1} \in \mathcal{S}$ with the current state $s_t \in \mathcal{S}$ and the action $a_t \in \mathcal{A}$, which means $s_{t+1} \sim \rho(\cdot|s_t, a_t.)$. The reward function $\mathcal{R} : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ quantifies the agent's performance in each transition. We use $r_t = \mathcal{R}(s_t, a_t)$ to denote the reward on each

transition. $\gamma \in [0, 1]$ is the discount factor trading off current and future rewards. Also, s_0 denotes the initial state.

The goal is to learn a policy that maximizes the total expected discounted rewards, i.e., $\pi^*(a_t|s_t) = \arg\max_\pi J(\pi)$ and $J(\pi) = \mathbb{E}[\sum_{t=0}^{T} \gamma^t r_t]$. Here T is the episode length. The policy π is the stochastic distribution of the action a_t taken by the agent under the state s_t, i.e., $a_t \sim \pi(\cdot|s_t.)$.

In order to implement RL algorithms, two value functions $V^\pi(s)$ and $Q^\pi(s, a)$ are defined. $V^\pi(s) = \mathbb{E}_{\tau\sim\pi}[\sum_{t=0}^{T} \gamma^t r_t|s_0 = s.]$ is the state-value function representing the expected discounted reward after state s with the policy π. $Q^\pi(s, a) = \mathbb{E}_{\tau\sim\pi}[\sum_{t=0}^{T} \gamma^t r_t|s_0 = s, a_0 = a.]$ is the action-value function representing the expected discounted reward after taking action a at state s with the policy π. Here, $\tau \sim \pi$ is the trajectory when applying π. From the definition, it is obvious that $V^\pi(s) = \mathbb{E}_{a\sim\pi} Q^\pi(s, a)$.

B. Adversarial Markov Decision Process

In the offline stage, we train the OFF-A and set up an adversary agent to mock the transfer gap from OFF-A to ON-A. This adversarial setting can be described as a two player discounted zero-sum Markov game [30], in which the protagonist and the adversary (opponent) are involved. In this article, we formalize this game as an adversarial MDP, which is an expansion of the MDP in Section II-A. In an AMDP, the adversary has an opposite goal to the protagonist and learns to hinder the protagonist by adjusting modelling errors. Note $(\cdot)_p$ or $(\cdot)^p$ to be the variables for the protagonist and $(\cdot)_o$ or $(\cdot)^o$ for the adversary.

Hence, AMDP can be expressed as the tuple $(\mathcal{S}, \mathcal{A}_p, \mathcal{A}_o, \rho, \mathcal{R}, \gamma)$. $\mathcal{A}_p$ is the action space of the protagonist representing the OFF-A, so $\mathcal{A}_p$ equals $\mathcal{A}$ in the MDP. $\mathcal{A}_o$ is the action space of the adversary, which mocks the transfer gap such as parameters errors. $\mathcal{S}$ and γ remain the same as MDP.

The definition of the other symbols in Section II-A are expanded respectively: $\rho : \mathcal{S} \times \mathcal{A}_p \times \mathcal{A}_o \to [0, \infty)$ is the state transition probability. At some time step t, the protagonist picks an action $a_t^p \sim \pi_p(\cdot|s_t)$ while the adversary chooses $a_t^o \sim \pi_o(\cdot|s_t)$. Then the environment transitions to the next state $s_{t+1} \sim \rho(\cdot|s_t, a_t^p, a_t^o)$.

The reward function $\mathcal{R}$ is expanded to $\mathcal{S} \times \mathcal{A}_p \times \mathcal{A}_o \to \mathbb{R}$ and $r_t = \mathcal{R}(s_t, a_t^p, a_t^o)$. In AMDP, the protagonist maximizes $J(\pi_p, \pi_o)$ with π_p while the adversary minimizes it using π_o as shown in eq. (1).

$$\begin{aligned} \pi_p^*\left(a_t^p|s_t\right) &= \arg\max_{\pi_p} \min_{\pi_o} J\left(\pi_p, \pi_o\right) \\ \pi_o^*\left(a_t^o|s_t\right) &= \arg\min_{\pi_o} \max_{\pi_p} J\left(\pi_p, \pi_o\right). \end{aligned} \tag{1}$$

C. VVC Problem Formulation

The VVC problem of ADNs is formulated as MDP for the online stage and ADMP for the offline stage. The specific definitions of episode, state space, action space and reward function are designed for both MDP and AMDP as follows.

1) Episode: In this article, we consider continuous steady-state control problems in both offline and online stage. An episode is defined to terminate at the grid failure situation, which determines the episode length $T \in [0, \infty)$.

2) State Space: The actual state space of the ADN is complex and impossible to fully perceive. Hence, in the MDP and AMDP, the state space $\mathcal{S}$ is supposed to be the same as observation space $\mathcal{O}$. $s \in \mathcal{S}$ is defined as a vector $\mathbf{s} = (\mathbf{P}, \mathbf{Q}, \mathbf{V}, t)$. Here $\mathbf{P}, \mathbf{Q}$ is the vector of nodal active/reactive power injections $P_i, Q_i(\forall i \in \mathcal{N})$, $\mathbf{V}$ is the vector of voltage magnitudes $V_i(\forall i \in \mathcal{N})$. t is the time step in each episode. All the states are assumed to be available in the studied distribution network. Similar state space and assumption are also seen in [20], [25].

3) Action Space: For the protagonist OFF-A in the offline stage and ON-A in the online stage, the action space $\mathcal{A} = \mathcal{A}_p$ includes the controllable range of the reactive power outputs of all IB-ERs and SVCs. Such devices provide high speed reactive power support and have large capacity in high penetration areas. Though other slow devices such as shunt capacitors do exist in ADNs, their operation schemes are usually scheduled offline because of their limited allowable daily switching times. Also, they could be coordinated with the proposed method in this article by rule-based regulation methods such as [12].

Hence, $\mathbf{a}_p = \mathbf{a} \in \mathcal{A}$ is defined as $(\mathbf{Q}_G, \mathbf{Q}_C)$, which is similar to [25]. $\mathbf{Q}_G$ and $\mathbf{Q}_C$ are the vectors of reactive power generation of IB-ERs and SVCs Q_{Gi}, Q_{Ci} respectively with range $|Q_{Gi}| \le \sqrt{S_{Gi}^2 - \overline{P_{Gi}}^2}$ and $\underline{Q_{Ci}} \le Q_{Ci} \le \overline{Q_{Ci}}$.

The action space of adversary in the offline stage is defined as the modeling errors represented by line parameter deviations. For $a_o \in \mathcal{A}_o$, $a_o = (\mathbf{\Delta r}_{ij}, \mathbf{\Delta x}_{ij}), (i, j) \in \mathcal{E}$ where $\mathcal{E}$ is the collection of branches. The simulation parameters are calculated with eq. (2),

$$\begin{aligned} r_{ij} &= r_{ij}^0 + \Delta r_{ij}, \quad \forall (i, j) \in \mathcal{E} \\ x_{ij} &= x_{ij}^0 + \Delta x_{ij}, \quad \forall (i, j) \in \mathcal{E} \end{aligned} \tag{2}$$

where r_{ij}, x_{ij} are resistance and reactance of branch ij, r_{ij}^0 and x_{ij}^0 are the approximate parameters.

The variation range of $\mathbf{\Delta r}_{ij}$ and $\mathbf{\Delta x}_{ij}$ determines the adversarial force in AMDP. Unreasonably strong adversary leads to very conservative OFF-A and make the offline adversarial training unstable. In ADN, the line parameter is calculated according to the type of wire and its length. Though such parameters are usually influenced by temperature,shape and manufacturing tolerance, their deviations are limited in reality.

4) Reward Function: Though the reward is generated by the environment without s involved explicitly in the classic RL algorithms, it is designed to be a function of previous states (observations). The reward function depends on the goal of VVC and is the key to achieve proper performance. In this article, the objectives are minimization of active power loss and mitigation of voltage violations. Hence, the reward consists of two terms: penalty for active power loss R_P, and penalty for voltage violations R_V.

R_P is calculated according to the active power injections $\mathbf{P}(t)$ as shown in eq. (3):

$$R_P(t) = -\sum_{i\in\mathcal{N}} P_i(t) \tag{3}$$

In this article, we focus the grid with high penetration of DGs, where the voltage violations are usually severe and the regulation capacity may be not enough to eliminate all violations in some scenarios. If the amount of voltage violation nodes is penalized [20], we cannot guarantee the violated voltage magnitudes in safe ranges and may lead to catastrophic violation nodes. Also, the voltage distance with 1 p.u. is not preferred [27] since it sacrifice the voltage space between $\underline{V}$ and $\overline{V}$. Hence, R_V is assigned according to the 2-norm of voltage magnitude violations called voltage violation rate (VVR) as $R_V(t) = \text{VVR}(t)$. ReLU is the well-known rectified linear unit function defined as $\text{ReLU} = \max(0, x)$. We have $R_V(t) \geq 0$ where the equality holds if and only if all voltage magnitudes satisfy the voltage constraints. Note that such R_V can effectively mitigate the voltage violations even when some violating nodes cannot be eliminated, as well as utilize the voltage space efficiently.

$$R_V(t) = -\sum_{i\in\mathcal{N}}\Big[\text{ReLU}^2\big(V_i(t)-\overline{V}\big)+\text{ReLU}^2\big(\underline{V}-V_i(t)\big)\Big] \tag{4}$$

The overall reward is the weighted sum of previous penalty terms, with a hyperparameter $C_v > 0$ as the weight ratio.

$$r_t = R(s_t, a_t, s_{t+1}) \doteq R_P(t) + C_v R_V(t). \tag{5}$$

III. Methods

A. Overview of the Two-Stage DRL-Based VVC

In this article, we propose a two-stage DRL-based VVC with a transferable DRL agent as shown in fig. 2. In the online stage, the state-of-art deep reinforcement learning algorithm soft actor-critic (SAC) is adopted for the ON-A due to its outstanding sample efficiency. However, the direct usage of SAC for online training often leads to severe security problems or unbearable cost in the start-up phase. Hence, we propose an alternative solution to train the transferable OFF-A in the offline stage which fully utilize the domain knowledge and historical experiences, and transfer OFF-A to ON-A to cope with the online training challenge.

Based on the off-policy feature of SAC that allows the target and behavior policy to be different, it is possible to use two common sources of samples: 1) offline models of ADNs, and 2) historical control samples. Both sources can only provide a rough description of the actual physical system since errors exist in the parameters of models and current system varies from the historical one over time. In this article, we focus on the former source since the latter one is not always available.

In this article, the OFF-A is trained against the possible modeling errors to be robust to the transfer gap or transferable in another way. The space of all possible modeling errors is termed the disturbance space $\mathcal{X}$. Instead of requiring the exact distribution of $\mathcal{X}$ and sampling all possible combinations [31], an adversary agent is trained jointly to mock the modeling errors and impede the protagonist by applying disturbances on the model parameters. The aim of the adversary is to disturb the protagonist in certain $\mathcal{X}$ and minimize the total discounted reward that the protagonist can get. Therefore, the adversary agent can learn the most tough scenarios for the protagonist and make the latter one as robust as possible regarding $\mathcal{X}$.

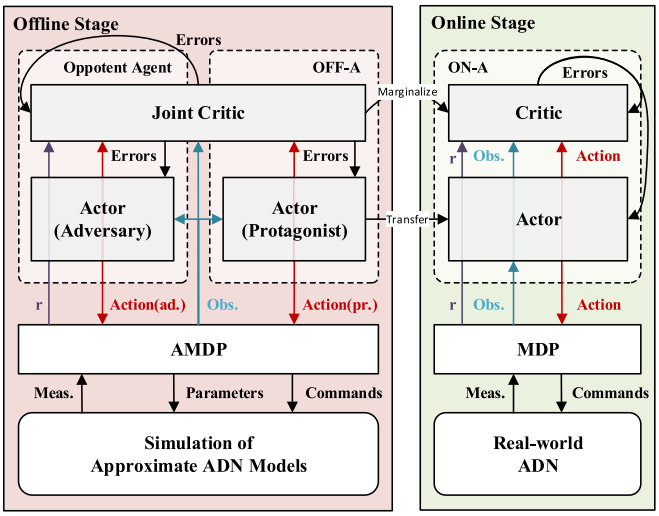


Fig. 2. The overall structure of the proposed two-stage VVC algorithm.

Though the original adversarial RL method [32] has been justified valid, it suffers poor efficiency and convergence in large scale cases. To improve the performance, we propose an adversarial reinforcement learning algorithm JASAC in Section III-C. Instead of training the protagonist and the adversary separately, JASAC shares the estimation of the value functions between the two agents, since they are in the same zero-sum game. The shared information allows each agent to optimize the value function approximation considering the other's behavior, so both the efficiency and convergence are promoted especially in cases with large scale of action and state space.

Finally, after the offline stage, the robust OFF-A is transferred to ON-A in the online stage. The numerical tests show that ON-A can realize VVC of the ADN safely and efficiently with continuous learning from online samples.

B. Soft Actor-Critic

In order to solve the MDPs, the actor-critic framework is popular among modern RL algorithms. Well-known actor-critic methods includes proximal policy optimization (PPO) [33], asynchronous advantage actor-critic (A3C) [34], deep deterministic policy gradient (DDPG) [35] and SAC [36], [37]. PPO and A3C are on-policy methods, which means the new samples have to be generated according to the latest policy of the agent. The sample efficiency of such methods is often far from satisfactory. Though DDPG is a famous off-policy method, it learns a deterministic policy and suffers in the online exploration.

To cope with these challenges, SAC follows the previous maximum entropy algorithms especially soft Q-learning (SQL) [38] and provides an off-policy maximum entropy RL algorithm. The state value function is entropy-regularized as

$$V^\pi(s) = \mathop{\mathbb{E}}_{\tau\sim\pi}\left[\sum_{t=0}^{\infty}\gamma^t(r_t+\alpha H(\pi(\cdot|s_t)))\middle| s_0 = s\right] \tag{6}$$

Algorithm 1: Soft Actor-Critic

Initialize experience pool, policy and value function approximators' parameter vectors;
repeat
  foreach *MDP step t* **do**
    $a_t \sim \pi(\cdot|s_t)$;
    Feed a_t to the environment, get reward r_t and next state s_{t+1};
    $\mathcal{D} \leftarrow \mathcal{D} \cup \{(s_t, a_t, r_t, s_{t+1})\}$;
    if $t \mod T_G$ **then**
      Sample a batch $B \in \mathcal{D}$ randomly;
      Update Q^π by eq. (7) on the batch B;
      Update π by eq. (8) on the batch B;
    end
  end
until *converge*;

where $H(\pi(\cdot|s_t)) = \mathbb{E}_{a\sim\pi(\cdot|s_t)}[-\log\pi(\cdot|s_t)]$ is the entropy for the stochastic policy at s_t. And $Q^\pi(s,a)$ is entropy-regularized accordingly as $Q^\pi(s,a) = \mathbb{E}_{\tau\sim\pi}[\sum_{t=0}^{T}\gamma^t r_t + \alpha\sum_{t=1}^{T}\gamma^t H(\pi(\cdot|s_t))|s_0 = s, a_0 = a]$.

During the learning process, all the samples of MDP are stored in the replay buffer $\mathcal{D}$ as $(s,a,r,s') \in \mathcal{D}$. To approximate Q^π by neural network iteratively, Bellman equation is applied to the entropy-regularized Q^π and approximated with samples as eq. (7) since $\mathbb{E}_{s',a'}H(\pi(\cdot|s')) = -\mathbb{E}_{s',a'}\log\pi(\cdot|s')$.

$$\begin{aligned} Q^\pi(s,a) &= \mathop{\mathbb{E}}_{s',a'}\left[R(s,a,s') + \gamma\left(Q^\pi(s',a') + \alpha H(\pi(\cdot|s'))\right)\right] \\ &\approx r + \gamma\left(Q^\pi(s',\tilde{a}') - \alpha\log\pi(\tilde{a}'|s')\right), \tilde{a}' \sim \pi(\cdot|s') \end{aligned} \tag{7}$$

Note that $\tilde{a}'$ is generated from the latest $\pi(\cdot|s')$ while s' is from $\mathcal{D}$. With this approximation, we use mean-squared Bellman error (MSBE) function to update the Q network.

With the value function as the "critic", the "actor" policy is optimized to maximize the state value function $V^\pi(s)$. In SAC, the reparameterization trick using a squashed Gaussian policy is introduced: $\tilde{a}_\theta(s,\xi) = \tanh(\mu_\theta(s) + \sigma_\theta(s)\odot\xi)$, $\xi \sim \mathcal{N}(0,\mathbf{I})$, where $\mu_\theta, \sigma_\theta$ are two parameterized neural networks. Hence, the expectation over actions is converted to the expectation over noise ξ, and the policy parameter θ can be optimized according to

$$\max_\theta \mathop{\mathbb{E}}_{\substack{s\sim\mathcal{D}\\ \xi\sim\mathcal{N}}}\left[Q(s,\tilde{a}_\theta(s,\xi)) - \alpha\log\pi_\theta(\tilde{a}_\theta(s,\xi)|s)\right] \tag{8}$$

A naive implementation of SAC is given in algorithm 1 using eqs. (7) and (8). Note that the update of agent is carried out every T_G steps, where $T_G \in \mathbb{N}^+$. Several techniques [37] to make SAC more practical and stable, such as optimization of the α, frozen target and two Q-functions, are omitted here. Also, the operator could stop the online learning at any satisfactory point and carry out the policy directly with $\xi = 0$ in $a = \tanh(\mu_\theta(s) + \sigma_\theta(s)\odot\xi)$.

C. Jointly Adversarial Soft Actor-Critic

In the offline stage, the protagonist and adversary agents are trained respectively to solve the AMDP in Section II-B. To realize the adversarial training, the original algorithm robust adversarial reinforcement learning (RARL) [32] alternatively trains the agents in a separate manner using policy gradient method. Though the effectiveness of RARL has been tested under several standard robotic environments, its stability, convergence and efficiency remain concerns. The bottlenecks for RARL are: 1) the separate training structure blocks the knowledge sharing between agents and make the algorithm less efficient; 2) policy gradient method is on-policy and suffers from stability and sample efficiency.

In this work, we innovate an algorithm called jointly adversarial soft actor-critic (JASAC) for the AMDP. An off-policy maximum entropy optimization is conducted based on SAC for each agent in an alternative manner. During the training process, the agents share the knowledge of the system using a spanned state-action value function and each optimizes the policy considering the other one on each step, which is the key novelty of JASAC.

Under the formulation of AMDP, we incorporate the maximum entropy framework like SAC for both agents. The value function is derived as

$$\begin{aligned} V^\pi(s) = \mathbb{E}\Bigg[\sum_{t=1}^{T}\gamma^{t-1}\big(\mathcal{R}(s_t, a_t^p, a_t^o) - \alpha_p\log\pi_p(a_t^p|s_t) \\ - \alpha_o\log\pi_o(a_t^o|s_t)\big)\Bigg] \end{aligned} \tag{9}$$

where π denotes (π_p, π_o), and α_p, α_o are the Lagrange multipliers for the entropy constraints. $\alpha_p > 0$ and $\alpha_o < 0$ since the adversary minimizes the expectation of total discounted reward as eq. (1).

1) Learning Q-Functions: In a separate training framework, each agent implements an approximator $Q_\phi(s,a)$ for the state-action value function. Though the action space of the agents are not the same, they are playing in the same game so that the actual value function $V^*(s)$ should be identical as $\max_{\pi_p}\min_{\pi_o} V^{\pi_p\pi_o}(s) = \min_{\pi_o}\max_{\pi_p} V^{\pi_p\pi_o}(s)$ [39]. The consistent value function inspires us to share the value approximation between agents during adversarial training and achieve the equilibrium faster. To do this, we define a joint Q-function instead of two separate ones as $Q^\pi : \mathcal{S}\times\mathcal{A}_p\times\mathcal{A}_o \to \mathbb{R}$ according to eq. (9).

Typically, we adapt Bellman equation to estimation the Q-value as shown in eq. (10). The shared information in Q^π allows us to consider both current policies π_p, π_o during the approximation instead of doing it alternatively.

$$\begin{aligned} Q^\pi(s,a_p,a_o) = \mathop{\mathbb{E}}_{s',a_p',a_o'}\Big[R(s,a_p,a_o,s') \\ + \gamma\Big(Q^\pi(s',a_p',a_o') - \alpha_p\log\pi_p(a_p'|s') \\ - \alpha_o\log\pi_o(a_o'|s')\Big)\Big] \end{aligned} \tag{10}$$

Also, the parameters of target Q networks $\hat{\phi}$ are delayed comparing to the learned ϕ as indicated in [36]. Hence, the

target of Q-functions $y(r, s')$ is calculated using the Bellman equation eq. (10) as

$$y(r,s') = r + \gamma\Big[Q_{\hat{\phi}}\Big(s', \tilde{a}'_p, \tilde{a}'_o\Big) - \alpha_p \log \pi\Big(\tilde{a}'_p|s'\Big) - \alpha_o \log \pi\big(\tilde{a}'_o|s'\big)\Big] \quad (11)$$

where $\tilde{a}'_p$ and $\tilde{a}'_o$) are generated using the latest $\pi_p(\cdot|s')$ and $\pi_o(\cdot|s')$. With this target value, we update ϕ_1, ϕ_2 by gradient descent on MSBE function $J_Q(\phi)$ which is usually calculated on a batch of samples B from $\mathcal{D}$:

$$J_Q(\phi) = \frac{1}{|B|}\sum_{s\in B}\Big[\big(Q_\phi(s,a) - y\big(r,s'\big)\big)^2\Big]. \quad (12)$$

2) Learning the Policy: The policies are learnt by optimizing the expected value. With the joint Q-functions, the value function $V^\pi(s)$ is calculated by the expectation on both action spaces.

$$V^\pi(s) = \mathop{\mathbb{E}}_{\substack{a_p\sim\pi_p\\ a_o\sim\pi_o}}\big[Q^\pi(s,a_p,a_o) - \alpha_p \log \pi_p\big(a_p|s\big) - \alpha_o \log \pi_o(a_o|s)\big] \quad (13)$$

To decouple the expectation and actions, the reparameterization trick in SAC is utilized here as $\tilde{a}_p^\theta(s,\xi_p) = \tanh(\mu_\theta(s) + \sigma_\theta(s)\odot\xi_p), \xi_p \sim \mathcal{N}(0,\mathbf{I})$ and $\tilde{a}_o^\omega(s,\xi_o) = \tanh(\mu_\omega(s) + \sigma_\omega(s)\odot\xi_o), \xi_o \sim \mathcal{N}(0,\mathbf{I})$, where θ is the parameter of protagonist policy and ω of adversary policy. The original problem $\max_{\pi_p}\min_{\pi_o} V^\pi(s_0)$ is transformed to

$$\max_\theta \min_\omega \mathop{\mathbb{E}}_{\substack{s\sim\mathcal{D}\\ \xi_p\sim\mathcal{N}\\ \xi_o\sim\mathcal{N}}}\Big[Q_\phi\Big(s, \tilde{a}_p^\theta\big(s,\xi_p\big), \tilde{a}_o^\omega(s,\xi_o)\Big) - \alpha_p \log \pi_p\Big(\tilde{a}_p^\theta\big(s,\xi_p\big)|s\Big) - \alpha_o \log \pi_o\big(\tilde{a}_o^\omega(s,\xi_o)|s\big)\Big] \quad (14)$$

Though it is possible in SAC to determine the optimal policy directly if the action space is discrete and small, evaluating the equilibrium solution in every step is time-consuming for a minimax optimization problem on continuous space. Following SAC and RARL, we alternatively learn π_p and π_o in a descent manner. Because θ and ω are independent in the entropy terms, we derive the loss function for policy parameters θ, ω as

$$J_{\pi_p}(\theta) = \frac{1}{|B|}\sum_{s\in B}\Big[Q_{\hat{\phi}}\Big(s, \tilde{a}_p^\theta\big(s,\xi_p\big), \tilde{a}_o^\omega(s,\xi_o)\Big) - \alpha_p \log \pi_p\Big(\tilde{a}_p^\theta\big(s,\xi_p\big)|s\Big)\Big] \quad (15)$$

$$J_{\pi_o}(\omega) = \frac{1}{|B|}\sum_{s\in B}\Big[Q_{\hat{\phi}}\Big(s, \tilde{a}_p^\theta(s,\xi_o), \tilde{a}_o^\omega(s,\xi_o)\Big) - \alpha_o \log \pi_o\big(\tilde{a}_o^\omega(s,\xi_o)|s\big)\Big] \quad (16)$$

where $B \in \mathcal{D}$. It should be emphasised that in eq. (15) and (16), the joint Q-functions provides the possible state value considering not only one's own action, but also the opponent's action. It makes the updated policy take the other's stochastic actions into consideration and correct the search direction.

Algorithm 2: Jointly Adversarial Soft Actor-Critic

Initialize experience pool, update frequency T_G, function approximators' parameter vectors θ, ω for policy and ϕ for value function;
repeat
  foreach *AMDP step t* **do**
    $a_t^p \sim \pi_p(\cdot|s_t), a_t^o \sim \pi_o(\cdot|s_t)$;
    Feed a_t^p, a_t^o to the environment, get reward r_t and next state s_{t+1};
    $\mathcal{D} \leftarrow \mathcal{D} \cup \{(s_t, (a_t^p, a_t^o), r_t, s_{t+1})\}$;
    if $t \mod T_G$ **then**
      Sample a batch $B \in \mathcal{D}$ randomly;
      Update the joint Q-function by:
      $$\phi \leftarrow \phi - \lambda\nabla_\phi J_Q(\phi)$$
      with $J_Q(\phi)$ from eq. (12);
      Update the protagonist policy by:
      $$\theta \leftarrow \theta + \lambda\nabla_\theta J_{\pi_p}(\theta)$$
      with J_{π_p} from eq. (15);
      Update the adversary policy by:
      $$\omega \leftarrow \omega - \lambda\nabla_\omega J_{\pi_o}(\omega)$$
      with J_{π_o} from eq. (16);
      $\hat{\phi} \leftarrow \eta\phi + (1-\eta)\hat{\phi}$;
    end
  end
until *converge*;
Marginalize Q_ϕ^* for protagonist as Q_Φ^* using eq. (17);
Output: Transferable protagonist state-action value function Q_Φ^* and π_p^* for online stage.

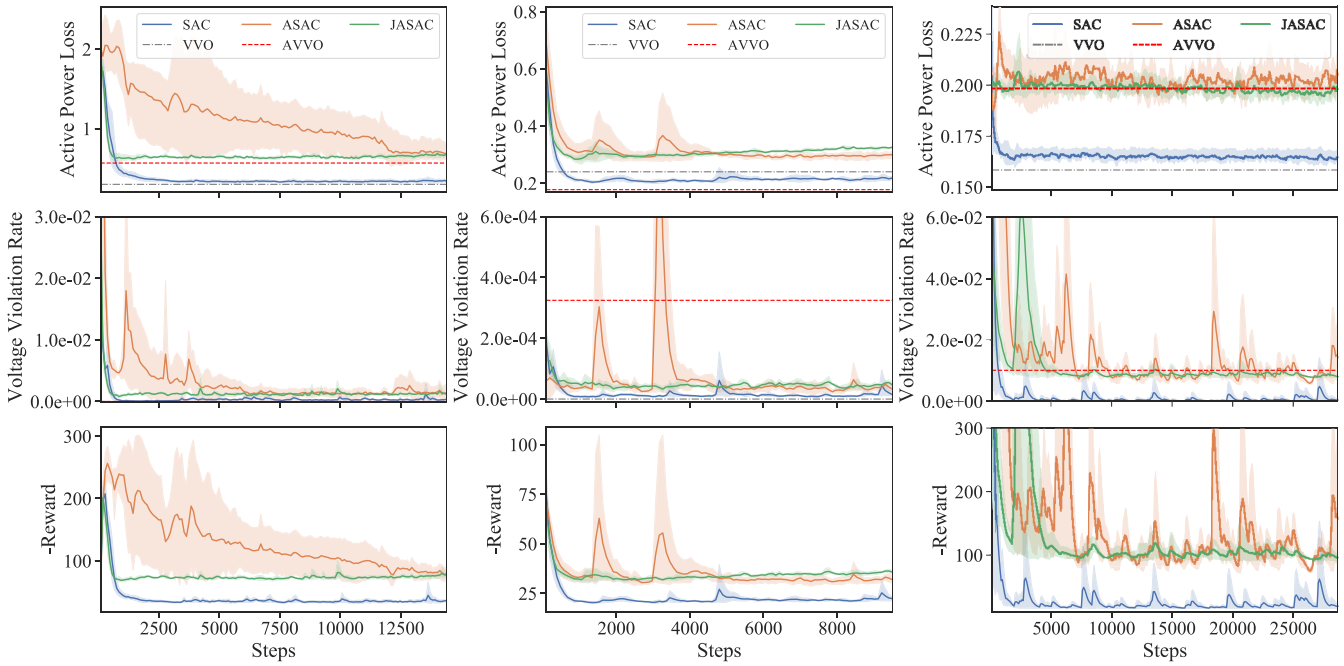
Finally, before migrating the protagonist to the online stage, we have to bridge JASAC and SAC. The protagonist policy π_p can be migrated directly as π in SAC. Because we want to continue the training online to realize optimality, Q_ϕ^* is marginalized for protagonist as Q_Φ^* since no adversary is conducted online.

$$Q_\Phi^*(s,a_p) = \mathop{\mathbb{E}}_{\xi_o\sim\mathcal{N}} Q_\phi^*\big(s,a_p,\tilde{a}_o^\omega(s,\xi_o)\big) \quad (17)$$

Practically, if the line status is changed due to blackout or maintenance, the agent in DRL-based methods should be reset and trained again for specific topology. In such situation, the OFF-A can be prepared before and immediately transferred to the online stage.

IV. Numerical Study

In this section, numerical experiments are conducted using 33-bus [40], 69-bus [41], and IEEE 123-bus [42] distribution test networks to validate the advantage of the proposed two-stage method over some popular benchmark algorithms including DRL algorithms and optimization-based algorithms. Steady-state distribution system reinforcement learning environments are built under the scheme of the well-known toolkit Gym [43] using the balanced power flow equations.



(a) Offline 33-bus case (b) Offline 69-bus case (c) Offline IEEE 123-bus case

Fig. 3. Offline training process experiment results for the 33-bus, 69-bus and IEEE 123-bus test cases.

In the 33-bus test network, 3 IB-ERs of 4.5 MVA capacity and 2.0 MW active power are connected to node 22,25,18 and 1 SVC of 4 MVA is connected to node 33. In the 69-bus test network, 4 IB-ERs of 3.5 MVA capacity and 1.2 MW active power are connected to node 6,9,31,41 and 2 SVCs of 3.5 MVA are connected to node 20,60. In the IEEE 123-bus test network, all spot loads are distributed equally on three phases and the elements at (1, 1) are used for 3-phase lines. 8 IB-ERs of 1.2 MVA capacity and 1.0 MW active power are connected to node 30,39,51,66,71,96,100,111 and 3 SVCs of 0.2 MVA is connected to node 25,54,76.

In all cases, all switches and regulators are set as the original state. All load and generation levels above are additionally multiplied with the fluctuation ratio extracted from a pilot project in eastern China to reflect the variance. The average ratio is 1.2 for the load profile and 2.5 for the generation profile. The smart inverters are assumed to follow IEEE 1547. In order to justify the efficiency of the proposed two-stage algorithm and adversarial training for transfer gap, we average the test results on 5 fixed sections of 96 steps to get rid of data noise. The voltage limitations are set to be [0.95, 1.05] in all cases. For the modeling errors (the difference between the online model and offline model), the ranges of parameters are set as [0.5, 2.0].

The reward parameters are chosen to be $C_v = 100$ for 33-bus case and $C_v = 1000$ for 69-bus case. In each simulation step, if the power flow does not converge or there is any $V_i > 1.15$ or $V_i < 0.85$, there is thought to be a grid failure and the episode is terminated with a negative reward $r = -300$.

A. Proposed and Baseline Algorithms Setup

In the offline stage, the proposed algorithm JASAC is implemented for solving AMDP. For the benchmark algorithm, we first follow the structure of RARL. The original version of RARL utilized on-policy algorithms to train both protagonist and adversarial agents and showed poor convergence in our environments. Hence, we substitute the on-policy algorithm for both agents with off-policy algorithms, which can be seen as a separately adversarial soft actor-critic (ASAC) algorithm. The major difference between JASAC and ASAC is that the former one shares critic value information between agents. An optimization-based algorithm with SOCP relaxation is implemented using the power flow model, either with oracle parameters (VVO) or with fake parameters (approximate VVO, AVVO). Also, in order to show the effectiveness of adversarial training, SAC is selected as one of the benchmarks to solve MDP.

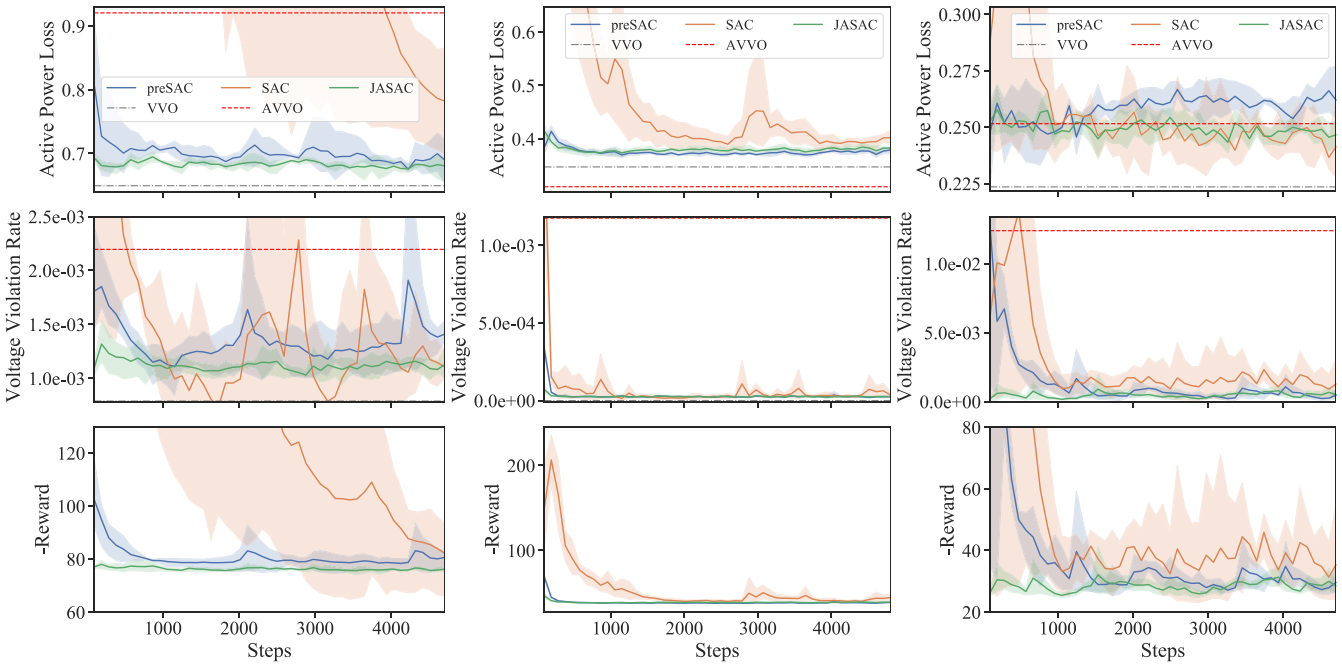
In the online stage, in addition to optimization-based and SAC benchmarks, we also migrate the trained SAC in the offline stage to the online (preSAC) following our two-stage DRL structure. The algorithm hyperparameters for both stages are listed in Section V.

All of the algorithms are implemented in Python, while the DRL-based algorithms utilize deep learning framework PyTorch, and the optimization-based methods utilize Casadi [44] and Ipopt. Experiments are run on a MacBook Pro with 16GB memory and 3.1GHz dual-core Intel i5 CPU.

Due to the stochastic property of DRL-based algorithms, we use 5 independent random seeds for each group of experiment, whose mean value and error bounds are presented in the figures. The results of three cases with two stages are shown in figs. 3 and 4 sequentially.

B. Offline Convergence and Efficiency

In the offline stage, we train the proposed JASAC algorithm and DRL-based benchmarks with our simulation



(a) Online 33-bus case (b) Online 69-bus case (c) Online IEEE 123-bus case

Fig. 4. Online Volt/VAR control process experiment results for the 33-bus, 69-bus and IEEE 123-bus test cases.

environments. Also, optimization-based benchmarks are evaluated with corresponding models respectively. The step average value of active power loss, voltage violation rate and negative reward (only for DRL algorithms) are shown in sub-figures of figs. 3a to 3c. The solid curves (green for JASAC, red for ASAC, blue for SAC) represent the mean performance across independent random seeds, and the light color filled regions are corresponding error bounds. The optimization-based benchmarks are drawn as black dashed lines.

The first important observation from figs. 3a to 3c is that SAC algorithm converges to a lower active power loss than the optimization-based method VVO without oracle parameters (the red dashed line), which significantly reveals the advantage of DRL-based algorithms over such parameter-sensitive optimization method regarding Volt/VAR control problem. Such fact could also be supported by the follow-up online results (figs. 4a to 4c and tables II and III) and other related works [20], [25]. On the other hand, though the oracle VVO attains the minimum of active power loss theoretically once given all true parameters, SAC algorithm could closely approach it after certain iterations, as depicted in the figure.

In terms of the comparison between JASAC/ASAC and SAC, we notice that the first two methods converge to the same value above SAC. Such gap arises due to the extra adversarial training mechanism in JASAC/ASAC, in which the adversary seeks for worse cases using model errors, essentially sacrificing a bit training performance for the robustness of the protagonist. In Section IV-C, we would further discuss that such design would help to produce trust-worth actions at the very start when dealing with the transfer gap from offline to online. Fortunately, such trade-off for robustness is almost of no cost, since in offline stage, our simulation has no actual loss on the real world system.

TABLE I
ALGORITHM HYPERPARAMETERS

Algo.	Parameter	Value
Shared	optimizer	Adam
	network type	feed-forward
	non-linearity	ReLU
	number of hidden layers	$\{2, 3, 3\}$
	hidden layer nuerons	$\{256, 256, 512\}$
	mini-batch size	$\{32, 64, 64\}$
	replay buffer size	4×10^5
	evaluation steps	96
	C_V	$100, 1000, 100$
	η	$0.9, 0.8, 0.7$
	λ	10^{-3}
SAC	α	$\{0.07, 0.03, 0.03\}$
	starting steps	300
preSAC	α	$\{0.07, 0.03, 0.03\}$
	starting steps	0
JASAC	α_p	$\{0.07, 0.03, 0.03\}$
	α_o	$\{0.04, 0.02, 0.02\}$
	starting steps	300
ASAC	α_p	$\{0.07, 0.03, 0.03\}$
	α_o	$\{0.04, 0.02, 0.02\}$
	starting steps	300

Finally, comparing JASAC with ASAC, the advantages of JASAC over ASAC with respect to training efficiency and convergence rate are evidently shown in figs. 3a to 3c. The third subplot, which illustrates the change of negative rewards with iterations, well reveals the dynamics of the training process. We could see that even with adversarial training, JASAC achieves similar efficiency and convergence rate as SAC, while ASAC suffers from the balancing process in the adversarial training. In fact, this significant improvement of JASAC compared to ASAC is credited to the shared value information

TABLE II
ONLINE PERFORMANCE COMPARISON WITH THE 33-BUS SYSTEM

	Algo.	P_{loss}/MW			VVR	
		Mean	Inc.*	Std.	Mean	Std.
Start	SAC	2.39e-00	1.74e-00	2.31e-01	3.58e-03	3.03e-03
	preSAC	8.08e-01	1.59e-01	1.08e-01	1.81e-03	5.18e-04
	JASAC	**6.93e-01**	**4.37e-02**	**6.94e-03**	**1.10e-03**	**8.41e-05**
	AVVO+	9.21e-01	2.71e-01	-	2.19e-03	-
Max	SAC	2.41e-00	1.76e-00	2.05e-01	1.08e-02	1.22e-02
	preSAC	8.28e-01	1.79e-01	7.87e-02	2.40e-03	6.54e-04
	JASAC	**7.00e-01**	**5.02e-02**	**3.23e-03**	**1.39e-03**	**1.34e-04**
	AVVO+	9.21e-01	2.71e-01	-	2.19e-03	-
Optimal Sol.o		6.49e-01	0	-	0	-

* mean value increment comparing to the lower bound.
+ optimization-based VVO with the offline approximate model.
o ideal optimal solution using perfect ADN model.

TABLE III
ONLINE PERFORMANCE COMPARISON WITH THE 69-BUS SYSTEM

	Algo.	P_{loss}/MW			VVR	
		Mean	Inc.*	Std.	Mean	Std.
Start	SAC	1.72e-00	1.37e-00	2.30e-01	1.74e-03	6.50e-04
	preSAC	3.84e-01	3.60e-02	1.89e-02	3.38e-04	5.02e-05
	JASAC	4.15e-01	6.75e-02	**1.38e-02**	**7.16e-05**	**3.31e-05**
	AVVO+	**3.11e-01**	**-3.7e-02**	-	1.18e-03	-
Max	SAC	1.96e-00	1.61e-00	3.83e-01	1.74e-03	6.50e-04
	preSAC	4.11e-01	6.71e-02	**1.28e-02**	3.38e-04	5.02e-05
	JASAC	4.15e-01	6.75e-02	1.38e-02	**7.31e-05**	**3.15e-05**
	AVVO+	**3.11e-01**	**-3.7e-02**	-	1.18e-03	-
Optimal Sol.o		3.48e-01	0	-	0	-

* mean value increment comparing to the lower bound.
+ optimization-based VVO with the offline approximate model.
o ideal optimal solution using perfect ADN model.

of the agents and the consideration of each other's policy. Such features make JASAC preferable in practice for Volt/VAR control, not only in this study but also in more complex potential tasks.

C. Online Safety and Efficiency

In the online stage, while maintaining similar simulation ideas as the offline stage, we change our focus to real world model, where simulations represent real world control process and actions are no longer cost-free. Hence, safety and efficiency become more important in every part of the simulations. Figures. 4a to 4c shows an example line plot and tables II to IV includes some major indices. In the table, we calculate the start and max value of active power loss and VVR. The ideal optimal solution calculated by VVO with oracle model and VVR $= 0$ is used as a baseline to calculate the increments in active power loss of other methods. Note that we trained the stochastic algorithms with 5 independent random seeds, based on which we could get the mean values and standard deviations across seeds.

First of all, comparing with the optimization-based VVO (the red dashed line), the DRL-based methods show obvious advantage on the voltage performance. In figs. 4a and 4c, the DRL-based methods also get lower active power loss. This observation supports the application of DRL-based methods rather than optimization-based ones in ADNs with incomplete models. Also, table V shows the online test time per step for JASAC, SAC and VVO, and validates the common superiority of DRL-based methods on test speed since they only need to calculate the neural networks instead of iterations.

The advantage of the proposed two-stage structure for DRL-based algorithms emerges in the online plot figs 4a to 4c and tables II to IV, where we could clearly observe a much better performance of JASAC and preSAC than SAC and VVO. For example, in fig. 4b and table III JASAC and preSAC have smaller VVR by an order of magnitude at the starting phase as well as at the maximum point. Also in fig. 4c and table IV, JASAC reduces the VVR by an order of magnitude at the starting phase comparing with VVO, which means better voltage performance at the very start. The rationale behind this is that the basic rules of the control tasks and the neural

TABLE IV
ONLINE PERFORMANCE COMPARISON WITH THE 123-BUS SYSTEM

	Algo.	P_{loss}/MW			VVR	
		Mean	Inc.*	Std.	Mean	Std.
Start	SAC	2.87e-01	6.30e-02	6.43e-02	6.09e-03	5.93e-03
	preSAC	**2.47e-01**	**2.39e-02**	2.11e-02	1.29e-02	1.13e-02
	JASAC	2.52e-01	2.90e-02	**7.36e-03**	**2.04e-04**	**1.94e-04**
	AVVO+	2.52e-01	2.80e-02	-	1.24e-02	-
Max	SAC	4.02e-01	1.78e-01	1.69e-01	1.81e-02	1.93e-02
	preSAC	2.72e-01	4.90e-02	**4.23e-03**	1.45e-02	8.91e-03
	JASAC	2.66e-01	4.24e-02	6.94e-03	**1.81e-03**	**7.66e-04**
	AVVO+	**2.52e-01**	**2.80e-02**	-	1.24e-02	-
Optimal Sol.o		2.24e-01	0	-	0	-

* mean value increment comparing to the lower bound.
+ optimization-based VVO with the offline approximate model.
o ideal optimal solution using perfect ADN model.

TABLE V
ONLINE TESTING TIME PER STEP

	33-bus	69-bus	123-bus
JASAC	0.91ms	1.10ms	1.31ms
SAC	0.90ms	1.09ms	1.32ms
VVO	27.0ms	71.0ms	122.4ms

network structures could be well learned in offline training stage and then inherited by the online stage. These consistent prior knowledge would make the algorithms take good actions even at the very beginning of the control process, thus giving rise to a significant improvement of the efficiency.

Further more, the adversarial training in the offline stage, as is mentioned previously, proves itself to be effective in the online stage when we look into the performances of preSAC and JASAC, especially at their starting phase. Specifically speaking in fig. 4a and table II, at the start of 33-bus case, preSAC has an active power loss increment of 1.59×10^{-1} from the ideal optimal solution, while JASAC cuts it by over 70% to 4.37×10^{-2}. JASAC also reduces the VVR by nearly 40% to 1.1×10^{-3}. For the max value in the process and other cases, such comparisons still hold. The key of our proposed JASAC lies in the strategy to prepare the agent for the transfer gap by first sacrificing a bit training performance in the

cost-free offline stage, in return for the robustness of the protagonist who could take preeminent actions in the real world online stage.

V. Conclusion

A two-stage deep reinforcement learning algorithm is proposed to optimize the reactive power distribution in inverter-based ADNs without accurate model parameters. The key novelty of the proposed algorithm lies in transfer of knowledge from the offline stage to online and significantly improvement on online control safety and efficiency. In the offline stage, we propose a novel RL algorithm JASAC with the formulation of AMDP to make the offline agent robust to the transfer gap. And in the online stage, OFF-A is transferred as ON-A and performs continuous online learning and control using SAC. Numerical studies on 33-bus, 69-bus, and IEEE 123-bus test networks have indicated that the proposed two-stage method outperforms the benchmark methods regarding safety and efficiency in the online stage, and also demonstrated that the proposed JASAC has better convergence and efficiency in the offline stage comparing to the benchmark adversarial learning algorithm.

In the future work, expanding the proposed adversarial framework and JASAC to both continuous value and discrete value devices in hierarchical structure is an interesting topic. Meta learning methods could be incorporated to deal with topology errors. The application of the proposed adversarial JASAC to more complex control problems such as coordinated active and reactive power control [45] is also a promising research direction.

Appendix A
Proof of Convergence for the Online Stage

In the online stage, the agent is transferred from the protagonist in offline adversarial learning. Then, the policy and value functions are updated continuously with online samples based on SAC. The proof of policy iteration convergence is provided in theorem 1 with lemmas lemma 1 proved in [36].

Lemma 1: For any π_{old}, let π_{new} be the optimized policy of the problem defined in eq. (8). Then $Q^{\pi_{\text{new}}}(s_t, a_t) \geq Q^{\pi_{\text{old}}}(s_t, a_t)$ for all $(s_t, a_t) \in \mathcal{S} \times \mathcal{A}$ with $|\mathcal{A}| < \infty$.

Theorem 1: Repeated application of policy update and value function update to any policy π converges to a policy π^* such that $Q^{\pi^*}(s_t, a_t) \geq Q^{\pi}(s_t, a_t)$ for all π and $(s_t, a_t) \in \mathcal{S} \times \mathcal{A}$.

Proof: From the definition of reward r_t, we have

$$r_t = -\sum_{i\in\mathcal{N}}\Big[P_i(t) + C_V\Big(\text{ReLU}^2\big(V_i(t)-\overline{V}\big)+\text{ReLU}^2\big(\underline{V}-V_i(t)\big)\Big)\Big]$$

According to the physical considerations, the terms can be bounded as

$$-\overline{P_D} \leq P_i(t) \leq \overline{P_G}$$
$$\sum_{i\in\mathcal{N}} P_i(t) \geq 0$$
$$0 \leq \text{ReLU}^2\big(V_i(t)-\overline{V}\big)+\text{ReLU}^2\big(\underline{V}-V_i(t)\big) \leq 1$$

where $\overline{P_{Di}}, \overline{P_{Gi}}$ are the maximum active power demand and generation of all nodes.

$$\therefore -N\big(\overline{P_{Gi}}+C_V\big) \leq r_t \leq 0$$

From the definition of Q,

$$Q^{\pi}(s,a) = \mathop{\mathbb{E}}_{\tau\sim\pi}\left[\sum_{t=0}^{T}\gamma^t r_t + \alpha\sum_{t=1}^{T}\gamma^t H(\pi(\cdot|s_t))\big|s_0=s, a_0=a\right]$$
$$Q^{\pi}(s,a) \leq \mathop{\mathbb{E}}_{\tau\sim\pi}\left[\alpha\overline{H}\frac{\gamma\big(1-\gamma^{T-1}\big)}{1-\gamma}\big|s_0=s, a_0=a\right] = \alpha\overline{H}\frac{\gamma\big(1-\gamma^{T}\big)}{1-\gamma}$$
$$Q^{\pi}(s,a) > -N\big(\overline{P_{Gi}}+C_V\big)\frac{1-\gamma^{T+1}}{1-\gamma}$$

where $0 < H(\pi) \leq \overline{H}$ since $|\mathcal{A}| < \infty$.

Hence $Q^{\pi}(s_t, a_t)$ is bounded. By lemma 1, the sequence Q^{π_i} is monotonically increasing, so the sequence converges to some π^*.

Suppose that we can find a π and a (s_t, a_t), s.t. $Q^{\pi^*}(s_t, a_t) < Q^{\pi}(s_t, a_t)$. Then according to lemma 1, we can get another $\pi_{\text{new}} = \pi$, s.t. $Q^{\pi_{\text{new}}}(s_t, a_t) > Q^{\pi^*}(s_t, a_t)$, which conflicts with the convergence of the sequence Q^{π_i}. Hence, the iteration converges to the optimal π^*. ■

Appendix B
Hyperparameters

The hyperparameters of SAC, JASAC, ASAC and preSAC used in this article are shown in table I.

Appendix C
Online Performance Quantification Tables

In order to quantify the starting stage of the cases, as well as to emphasize the worst case during the online control process, the indices are read from the figures figs. 4a to 4c. The best performance among methods are marked as bold in each column. Though preSAC and VVO are better regarding active power loss in some cases, JASAC has the best voltage performance in all cases, which are most concerned in ADNs.

References

[1] S. Deshmukh, B. Natarajan, and A. Pahwa, "Voltage/VAR control in distribution networks via reactive power injection through distributed generators," *IEEE Trans. Smart Grid*, vol. 3, no. 3, pp. 1226–1234, Sep. 2012.

[2] A. R. Malekpour and A. Pahwa, "Reactive power and voltage control in distribution systems with photovoltaic generation," in *Proc. North Amer. Power Symp. (NAPS)*, Champaign, IL, USA, Sep. 2012, pp. 1–6.

[3] T. Xu and W. Wu, "Accelerated ADMM-based fully distributed inverter-based volt/var control strategy for active distribution networks," *IEEE Trans. Ind. Informat.*, vol. 16, no. 12, pp. 7532–7543, Dec. 2020.

[4] M. Farivar, C. R. Clarke, S. H. Low, and K. M. Chandy, "Inverter VAR control for distribution systems with renewables," in *Proc. IEEE Int. Conf. Smart Grid Commun. (SmartGridComm)*, Brussels, Belgium, Oct. 2011, pp. 457–462.

[5] M. B. Liu, C. A. Canizares, and W. Huang, "Reactive power and voltage control in distribution systems with limited switching operations," *IEEE Trans. Power Syst.*, vol. 24, no. 2, pp. 889–899, May 2009.

[6] A. Borghetti, "Using mixed integer programming for the volt/var optimization in distribution feeders," *Elect. Power Syst. Res.*, vol. 98, pp. 39–50, May 2013. [Online]. Available: http://www.sciencedirect.com/science/article/pii/S0378779613000138

[7] S. Auchariyamet and S. Sirisumrannukul, "Optimal daily coordination of volt/VAR control devices in distribution systems with distributed generators," in *Proc. 45th Int. Univ. Power Eng. Conf. (UPEC)*, Cardiff, U.K., Aug. 2010, pp. 1–6.

[8] H. Zhu and H. J. Liu, "Fast local voltage control under limited reactive power: Optimality and stability analysis," *IEEE Trans. Power Syst.*, vol. 31, no. 5, pp. 3794–3803, Sep. 2016.

[9] P. Jahangiri and D. C. Aliprantis, "Distributed volt/VAR control by PV inverters," *IEEE Trans. Power Syst.*, vol. 28, no. 3, pp. 3429–3439, Aug. 2013.

[10] M. Manbachi *et al.*, "Real-time co-simulation platform for smart grid volt-VAR optimization using IEC 61850," *IEEE Trans. Ind. Informat.*, vol. 12, no. 4, pp. 1392–1402, Aug. 2016.

[11] H. J. Liu, W. Shi, and H. Zhu, "Distributed voltage control in distribution networks: Online and robust implementations," *IEEE Trans. Smart Grid*, vol. 9, no. 6, pp. 6106–6117, Nov. 2018.

[12] W. Zheng, W. Wu, B. Zhang, H. Sun, and Y. Liu, "A fully distributed reactive power optimization and control method for active distribution networks," *IEEE Trans. Smart Grid*, vol. 7, no. 2, pp. 1021–1033, Mar. 2016.

[13] W. Zheng, W. Wu, B. Zhang, and Y. Wang, "Robust reactive power optimisation and voltage control method for active distribution networks via dual time-scale coordination," *IET Gener. Transm. Distrib.*, vol. 11, no. 6, pp. 1461–1471, Apr. 2017.

[14] F. U. Nazir, B. C. Pal, and R. A. Jabr, "A two-stage chance constrained volt/var control scheme for active distribution networks with nodal power uncertainties," *IEEE Trans. Power Syst.*, vol. 34, no. 1, pp. 314–325, Jan. 2019.

[15] H. Zhao, Q. Wu, J. Wang, Z. Liu, M. Shahidehpour, and Y. Xue, "Combined active and reactive power control of wind farms based on model predictive control," *IEEE Trans. Energy Convers.*, vol. 32, no. 3, pp. 1177–1187, Sep. 2017.

[16] Y. Yang and W. Wu, "A distributionally robust optimization model for real-time power dispatch in distribution networks," *IEEE Trans. Smart Grid*, vol. 10, no. 4, pp. 3743–3752, Jul. 2019.

[17] J. R. Marden, S. D. Ruben, and L. Y. Pao, "A model-free approach to wind farm control using game theoretic methods," *IEEE Trans. Control Syst. Technol.*, vol. 21, no. 4, pp. 1207–1214, Jul. 2013.

[18] P. M. O. Gebraad, F. C. van Dam, and J. van Wingerden, "A model-free distributed approach for wind plant control," in *Proc. Amer. Control Conf.*, Washington, DC, USA, Jun. 2013, pp. 628–633.

[19] D. B. Arnold, M. Negrete-Pincetic, M. D. Sankur, D. M. Auslander, and D. S. Callaway, "Model-free optimal control of VAR resources in distribution systems: An extremum seeking approach," *IEEE Trans. Power Syst.*, vol. 31, no. 5, pp. 3583–3593, Sep. 2016.

[20] W. Wang, N. Yu, Y. Gao, and J. Shi, "Safe off-policy deep reinforcement learning algorithm for volt-VAR control in power distribution systems," *IEEE Trans. Smart Grid*, vol. 11, no. 4, pp. 3008–3018, Jul. 2020.

[21] V. Mnih *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[22] D. Silver *et al.*, "Mastering the game of Go with deep neural networks and tree search," *Nature*, vol. 529, no. 7587, pp. 484–489, 2016.

[23] X. Pan, Y. You, Z. Wang, and C. Lu, "Virtual to real reinforcement learning for autonomous driving," 2017. [Online]. Available: arXiv:1704.03952.

[24] Y. Duan, X. Chen, R. Houthooft, J. Schulman, and P. Abbeel, "Benchmarking deep reinforcement learning for continuous control," in *Proc. Int. Conf. Mach. Learn.*, 2016, pp. 1329–1338.

[25] C. Li, C. Jin, and R. Sharma, "Coordination of PV smart inverters using deep reinforcement learning for grid voltage regulation," in *Proc. 18th IEEE Int. Conf. Mach. Learn. Appl. (ICMLA)*, 2019, pp. 1930–1937.

[26] W. Wang, N. Yu, J. Shi, and Y. Gao, "Volt-VAR control in power distribution systems with deep reinforcement learning," in *Proc. IEEE Int. Conf. Commun. Control Comput. Technol. Smart Grids (SmartGridComm)*, Beijing, China, Oct. 2019, pp. 1–7.

[27] Q. Yang, G. Wang, A. Sadeghi, G. B. Giannakis, and J. Sun, "Two-timescale voltage control in distribution grids using deep reinforcement learning," *IEEE Trans. Smart Grid*, vol. 11, no. 3, pp. 2313–2323, May 2020.

[28] H. Xu, A. D. Domínguez-García, and P. W. Sauer, "Optimal tap setting of voltage regulation transformers using batch reinforcement learning," *IEEE Trans. Power Syst.*, vol. 35, no. 3, pp. 1990–2001, May 2020.

[29] J. Duan *et al.*, "Deep-reinforcement-learning-based autonomous voltage control for power grid operations," *IEEE Trans. Power Syst.*, vol. 35, no. 1, pp. 814–817, Jan. 2020.

[30] M. L. Littman, "Markov games as a framework for multi-agent reinforcement learning," in *Machine Learning Proceedings 1994*, W. W. Cohen and H. Hirsh, Eds. San Francisco, CA, USA: Morgan Kaufmann, 1994, pp. 157–163. [Online]. Available: http://www.sciencedirect.com/science/article/pii/B9781558603356500271

[31] A. Rajeswaran, S. Ghotra, B. Ravindran, and S. Levine, "EPOpt: Learning robust neural network policies using model ensembles," 2016. [Online]. Available: arXiv:1610.01283.

[32] L. Pinto, J. Davidson, R. Sukthankar, and A. Gupta, "Robust adversarial reinforcement learning," in *Proc. 34th Int. Conf. Mach. Learn. Vol. 70*, 2017, pp. 2817–2826.

[33] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017. [Online]. Available: https://arxiv.org/abs/1707.06347.

[34] V. Mnih *et al.*, "Asynchronous methods for deep reinforcement learning," in *Proc. Int. Conf. Meach. Learn.*, 2016, pp. 1928–1937.

[35] D. Silver, G. Lever, N. Heess, T. Degris, D. Wierstra, and M. Riedmiller, "Deterministic policy gradient algorithms," in *Proc. 31st Int. Conf. Mach. Learn. (ICML)*, vol. 1, Jun. 2014, pp. 387–395.

[36] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," 2018. [Online]. Available: arXiv:1801.01290.

[37] T. Haarnoja, S. Ha, A. Zhou, J. Tan, G. Tucker, and S. Levine, "Learning to walk via deep reinforcement learning," 2018. [Online]. Available: https://arxiv.org/abs/1812.11103.

[38] T. Haarnoja, H. Tang, P. Abbeel, and S. Levine, "Reinforcement learning with deep energy-based policies," in *Proc. 34th Int. Conf. Mach. Learn. Vol. 70*, 2017, pp. 1352–1361.

[39] J. Grau-Moya, F. Leibfried, and H. Bou-Ammar, "Balancing two-player stochastic games with soft Q-learning," 2018. [Online]. Available: arXiv:1802.03216.

[40] M. E. Baran and F. F. Wu, "Network reconfiguration in distribution systems for loss reduction and load balancing," *IEEE Trans. Power Del.*, vol. 4, no. 2, pp. 1401–1407, Apr. 1989.

[41] D. Das, "Optimal placement of capacitors in radial distribution system using a fuzzy-ga method," *Int. J. Elect. Power Energy Syst.*, vol. 30, nos. 6–7, pp. 361–367, 2008. [Online]. Available: http://www.sciencedirect.com/science/article/pii/S0142061507001342

[42] K. P. Schneider *et al.*, "Analytic considerations and design basis for the IEEE distribution test feeders," *IEEE Trans. Power Syst.*, vol. 33, no. 3, pp. 3181–3188, May 2018.

[43] G. Brockman *et al.*, "OpenAI gym," 2016. [Online]. Available: https://arxiv.org/abs/1606.01540.

[44] J. A. E. Andersson, J. Gillis, G. Horn, J. B. Rawlings, and M. Diehl, "CasADi: A software framework for nonlinear optimization and optimal control," *Math. Program. Comput.*, vol. 11, no. 1, pp. 1–36, 2019.

[45] H. Nazaripouya, H. R. Pota, C. Chu, and R. Gadh, "Real-time model-free coordination of active and reactive powers of distributed energy resources to improve voltage regulation in distribution systems," *IEEE Trans. Sustain. Energy*, vol. 11, no. 3, pp. 1483–1494, Jul. 2020.

Haotian Liu (Graduate Student Member, IEEE) received the B.S. degree from the Electrical Engineering Department, Tsinghua University, Beijing, China, where he is currently pursuing the Ph.D. degree. His research interests include active distribution system operation and control, model-free control, and machine learning especially reinforcement learning and their applications in the energy system.

Wenchuan Wu (Fellow, IEEE) received the B.S., M.S., and Ph.D. degrees from the Electrical Engineering Department, Tsinghua University, Beijing, China. He is currently a Professor with Tsinghua University and the Deputy Director of the State Key Laboratory of Power Systems. He has published 400 peer-reviewed papers including 66 IEEE journal papers. His research interests include energy management system, active distribution system operation and control, and machine learning and its application in energy system. He was a recipient of the National Science Fund of China Distinguished Young Scholar Award in 2017.